

Lab 3: DDP / DALI Implementation & Profiling

CSED490F Deep Learning Implementation — Team 4

20262756 Kwangmo Yang (양광모), 20262346 Chanyoung Gwak (곽찬영)

1 Implementation

All changes are in `handler/` and `scripts/launch.sh`; `train_cifar.py` is untouched. Global batch sizes stay at 512 (train/test) in every mode, so DP, DDP and DALI do the same work per iteration.

P0 (Nsight command). The scaffold command is wrapped with `nsys profile`. NVTX is required for the `nvtx.annotate` ranges in the code; CPU sampling is disabled because Vast.ai containers are unprivileged. Children created by `mp.spawn` are traced into the same report.

```
CUDA_VISIBLE_DEVICES=$LOCAL_GPU_IDS nsys profile --trace=cuda,nvtx,osrt,cudnn,cublas \  
--cuda-memory-usage=true --sample=none --cpuctxsw=none --force-overwrite=true \  
--output="$NSIGHT_LOG_DIR/$NSIGHT_FILE_NAME" $NSYS_EXTRA_ARGS python train_cifar.py ...
```

P1–P3 (process group). `run_process` calls `mp.spawn(func, args=(args,), nprocs=num_gpu)`, so `main_func(proc_id, args)` runs once per GPU. `initialize_group` first calls `torch.cuda.set_device(proc_id)` (GPUs are restricted by `CUDA_VISIBLE_DEVICES`, so `rank = local device`) and then `dist.init_process_group("nccl", init_method="tcp://host:port", world_size, rank)`; setting the device first binds each NCCL communicator to its own GPU. `destroy_process` calls `dist.destroy_process_group()` only if the group was initialized, because it runs in a `finally` block.

P4 (model). Unlike DP (one process driving all GPUs, `device_ids=[0..N-1]`), each DDP process owns one GPU: `model.cuda(dev)` and `DDP(model, device_ids=[dev], output_device=dev)` with `dev = torch.cuda.current_device()`.

P5 (DDP loader). Each split gets a `DistributedSampler(num_replicas=world_size, rank=rank)` (shuffled for train, not for test), passed as `sampler=` with `shuffle` removed from the `DataLoader`. The per-process batch is `batch // world_size` (asserted divisible), so ranks see disjoint shards and the global batch equals DP's. The metrics in `DDP/train.py` are all-reduced, so logged loss/accuracy are global.

P6 (DALI pipeline). The pipeline reproduces the torchvision transforms operator by operator:

torchvision (DP/DDP)	DALI (CifarPipeline.define_graph)
ImageFolder read, shuffle	<code>fn.readers.file(file_root, shard_id=rank, num_shards=N, random_shuffle=is_train, name="Reader")</code>
PIL decode	<code>fn.decoders.image(device="mixed", output_type=RGB) → GPU, HWC uint8</code>
RandomCrop(32, padding=4)	<code>fn.paste(ratio=1.25, fill_value=0) (centered, 40×40) + CMN crop=(32,32), random crop_pos_x/y</code>
RandomHorizontalFlip()	<code>mirror = fn.random.coin_flip(0.5) passed to CMN</code>
ToTensor + Normalize	<code>CMN mean/std (×255), dtype=FLOAT, output_layout="CHW"</code>
Cutout(16) (after normalize)	<code>provided fn_dali_cutout (fn.erase, fill 0)</code>

`CMN = fn.crop_mirror_normalize`. The seed is `12345 + shard_id` so ranks draw different augmentations. The class folders are sorted alphabetically, which matches torchvision's CIFAR-10 label order. Two known differences: the provided cutout keeps the square fully inside the image, and PNG (unlike JPEG) is decoded on the CPU even with the mixed backend.

P7 (DALI loader). Per split, a `CifarPipeline` is built with `batch batch // world_size`, `device_id = current device`, and `shard_id = rank`, then wrapped as `DALIWrapper(DALIGenericIterator(pipe, ["data","label"], reader_name="Reader", last_batch_policy=PARTIAL, auto_reset=True))`. `PARTIAL` matches `drop_last=False`; `auto_reset` is needed because the wrapper re-raises `StopIteration`.

Correctness. On 2 GPUs (98 iterations each), DDP and DDP+DALI reach the same 1-epoch train loss (1.7676 vs. 1.7675) and a similar test accuracy (48.5% vs. 47.6%), so the DALI pipeline is equivalent to the torchvision one.

2 Setup and method

We use the TA-provided logs (RTX 3090, 1/2/4 GPUs, all three modes) as the main data, because only they contain hardware GPU metrics (*SMs Active* and *DRAM bandwidth*; collecting them needs privileges that Vast.ai containers do not have). Our own runs (Vast.ai, 2×RTX 3090, no GPU peer-to-peer access, 1/2 GPUs) reproduce the trends (Table 2). Each `.nsys-rep` was exported to SQLite and summarized with `report/analyze.py`. All numbers are inside the NVTX Epoch 0 range and averaged over ranks. We split one iteration (98 per epoch) using the NVTX rows:

- **batch range:** the NVTX Batch i / Train batch range, split into **forward** and **backward** (+ upload, loss);
- **outside batch:** iteration time minus the batch range, i.e. `next(loader)` plus the metric `all-reduce/.item()`, which also waits for queued GPU work.

NVTX ranges measure CPU time. We therefore also report *compute busy*: the union of non-NCCL kernel intervals over the epoch, per GPU. NCCL kernels are excluded because they stay resident while waiting for other ranks.

3 Results

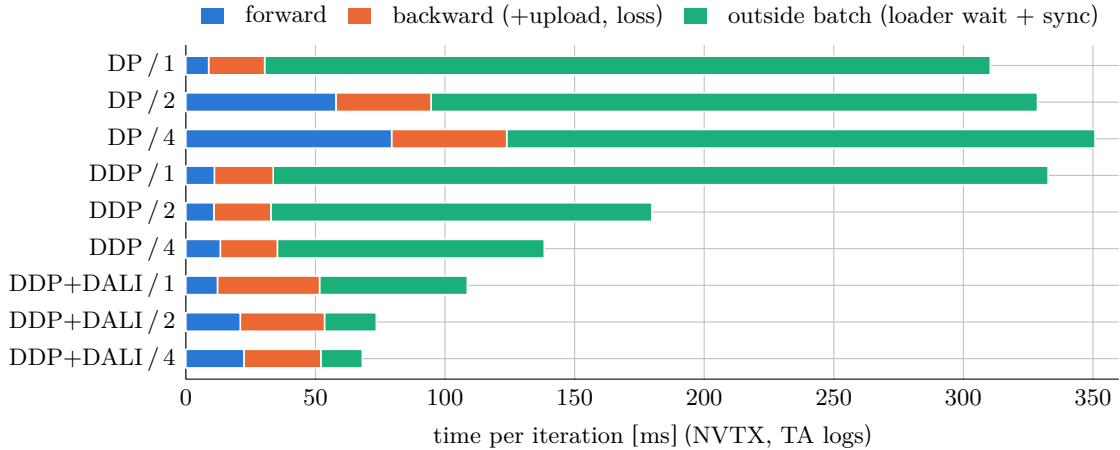


Figure 1: Per-iteration breakdown from the NVTX rows (mode / #GPU). Values are in Table 1.

Table 1: NVTX breakdown and GPU metrics (TA logs, mean over GPUs). Busy = compute-kernel busy time over the epoch.

Mode	GPU	NVTX [ms / iter]					GPU [%]			per GPU	
		epoch [s]	iter	fwd	bwd	outside	SMs act.	DRAM R/W	busy	kernel ms/it	NCCL [s]
DP	1	30.4	310	8.8	20.9	280	28.0	9.1 / 5.3	31.1	96.4	–
	2	32.2	328	57.9	35.9	234	14.1	4.4 / 2.6	15.9	52.3	0.98
	4	34.4	351	79.4	43.1	227	6.8	2.2 / 1.3	7.8	27.5	2.50
DDP	1	32.6	333	11.0	22.1	299	26.3	8.5 / 5.0	29.2	97.0	–
	2	17.7	180	10.8	21.4	147	24.8	8.0 / 4.8	28.3	51.1	2.18
	4	13.6	138	13.3	21.3	103	17.0	5.5 / 3.5	22.4	31.0	4.01
DDP+DALI	1	10.6	109	12.2	38.7	57	82.0	25.7 / 15.1	91.1	99.0	–
	2	7.2	74	21.0	31.6	20	61.2	19.3 / 11.6	70.2	51.7	1.81
	4	6.7	69	22.4	28.9	16	34.1	11.0 / 7.0	45.9	31.5	3.44

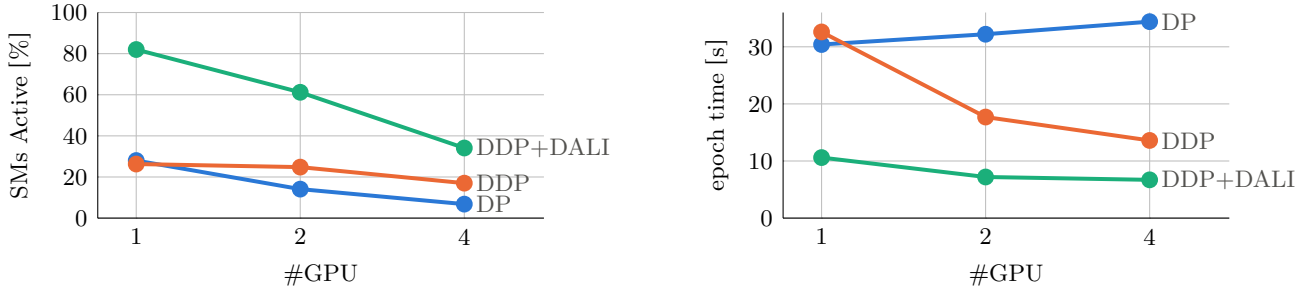


Figure 2: GPU utilization (SMs Active, left) and epoch time (right) vs. number of GPUs (TA logs, values in Table 1).

Table 2: Our Vast.ai runs (nvidia-smi sampled every 200 ms inside the epoch; memory = peak allocated / nvidia-smi max, per GPU).

Mode	GPU	epoch [s]	iter [ms]	outside [ms]	busy [%]	smi GPU/mem [%]	mem. [MiB]	H→D [MiB]
DP	1	27.8	284	259	37.9	40 / 21	3544 / 3854	586
DP	2	33.4	341	261	14.9	44 / 8	1939 / 2312	881
DDP	1	27.7	282	252	38.2	37 / 18	3662 / 4022	586
DDP	2	18.2	186	158	38.4	48 / 15	1957 / 2378	586
DDP+DALI	1	11.7	119	90	93.9	92 / 45	4086 / 4552	158
DDP+DALI	2	9.7	99	65	74.8	90 / 28	2389 / 2748	158

4 Analysis: how and why GPU/memory utilization changes

Baseline (1 GPU). With the torchvision loader (`num_workers=0`), the *outside-batch* time is 280–300 ms of a 310–333 ms iteration. Our logs confirm that this is data loading: the loader’s own `Data` timer is 0.17 s per iteration. PNG decode and augmentation of 512 images run on one CPU thread, while the GPU needs only ≈ 97 ms of kernels per iteration. The GPU is therefore idle about 70% of the time (SMs Active 26–28%, DRAM read 9%). Both DP and DDP start from this CPU-bound point.

DP gets slower with more GPUs. Kernel time per GPU halves ($96 \rightarrow 52 \rightarrow 28$ ms), but utilization falls roughly as $1/N$ (SMs Active $28 \rightarrow 14 \rightarrow 7\%$, DRAM read $9 \rightarrow 4 \rightarrow 2\%$) and the epoch grows ($30.4 \rightarrow 34.4$ s). Two reasons: (i) a single process still loads all 512 images per iteration, so the *outside-batch* time stays at 227–280 ms and extra GPUs only wait longer. (ii) The DP work inside the batch grows: **forward** goes from 8.8 to 79.4 ms, because every iteration scatters the input, replicates the parameters (`ncclBroadcast`), runs the replicas from Python threads under the GIL, and gathers the outputs on GPU 0; **backward** goes from 21 to 43 ms because of the gradient `ncclReduce`. The input also crosses PCIe twice: H→D grows from 586 to 881/1029 MiB and D→H from 0 to 295/442 MiB, exactly $(N-1)/N$ of the batch, because GPU 0 scatters through host memory. This is the communication overhead the TAs warned about; it grows with N while the useful work per GPU shrinks. Memory per GPU also halves ($3544 \rightarrow 1939$ MiB allocated), since each replica holds only $512/N$ activations.

DDP scales with the number of processes. DDP runs one process per GPU, so each loads only its $512/N$ shard in parallel. The *outside-batch* time falls $299 \rightarrow 147 \rightarrow 103$ ms and the epoch $32.6 \rightarrow 17.7 \rightarrow 13.6$ s ($2.4\times$ on 4 GPUs). The batch range stays at ≈ 33 ms. Parameters are synchronized once (`Sync model parameters`); per step only the small BatchNorm buffers are broadcast (all `ncclBroadcast` calls fall inside **forward**), with no input scatter or model replication as in DP. Gradients are all-reduced in 2–3 buckets that overlap with **backward**: 544 of the 1368 `ncclAllReduce` calls (2 GPUs) fall inside **backward**, and most of the rest are the per-step metric `reduce_tensor` outside the batch range. As a result, utilization per GPU is roughly kept at 2 GPUs (SMs $26 \rightarrow 25\%$, DRAM $8.5 \rightarrow 8.0\%$). At 4 GPUs it drops to 17% because the per-GPU compute (31 ms) shrinks faster than the loader time, and NCCL kernels (4.0 s per GPU) spend more time waiting for the slowest rank. The bucket copies show up as 4.2 GiB of D→D traffic per GPU per epoch. Peak memory per GPU is the same as DP’s (1957 MiB on 2 GPUs) and balanced across ranks.

DALI removes the loader bottleneck. DALI prefetches batches and runs crop/flip/normalize/cutout on the GPU, overlapping them with training. On 1 GPU the *outside-batch* time falls from 299 to 57 ms

and the epoch from 32.6 to 10.6 s ($3.1\times$). The GPU becomes busy (SMs Active 82%, compute busy 91%, nvidia-smi 92%) and memory bandwidth use triples (DRAM read 26%, nvidia-smi memory util 45% vs. 18%). Most of the remaining outside-batch time is now `.item()` waiting for queued kernels, not data. H→D traffic drops $3.7\times$ ($586\rightarrow158\text{--}172$ MiB), because uint8 images are uploaded instead of normalized float32 tensors. The costs: the pipeline build makes `Set data loader` 3.3 s instead of 1.0 s, and DALI buffers use ≈ 430 MiB more GPU memory (4086 vs. 3662 MiB). Multi-GPU scaling now hits another limit. Per-GPU kernel time falls to 31 ms on 4 GPUs, but the CPU-side batch range stays at ≈ 52 ms, so launching ResNet-18’s kernels for a small batch of 128 dominates. SMs Active drops to 61% (2 GPUs) and 34% (4 GPUs), and the epoch improves only $10.6\rightarrow 7.2\rightarrow 6.7$ s.

Notes. (1) nvidia-smi’s *GPU util* counts any running kernel, including NCCL kernels waiting for peers. For our DP 2-GPU run it reports 44% while compute kernels are busy only 15%, so we use SMs Active and compute-busy time. (2) The TA logs report *GPU Active* = 100% on some GPUs with the same SMs Active as the others, so we do not use that metric. (3) Our instance has no GPU peer-to-peer access, and its per-GPU kernel time for DDP on 2 GPUs is higher (71 vs. 51 ms). The trends and the ranking $DP < DDP < DDP+DALI$ are the same as in the TA logs.

Conclusion. DP is limited by a single-process data loader and by per-iteration scatter/replicate/gather, so adding GPUs lowers utilization and slows training. DDP parallelizes loading across processes and overlaps gradient all-reduce with backward, which gives near-linear speedup while loading dominates. DALI moves augmentation to the GPU and prefetches asynchronously, which raises GPU and memory-bandwidth utilization to 80–90% on one GPU. After that, the next bottleneck is CPU kernel-launch overhead for small per-GPU batches.